

Laporan Tugas Besar Milestone 2

IF2224 Teori Bahasa Formal dan Otomata

Syntax Analysis

Semester II Tahun 2025/2026



Disusun oleh:

13524041 - Nathan Adhika Santosa

13524053 - Muhammad Haris Putra Sulastianto

13524085 - Ariel Cornelius Sitorus

13524091 - Vara Azzara Ramli Pulukadang

Laboratorium Ilmu Rekayasa dan Komputasi

Sekolah Teknik Elektro dan Informatika

Program Studi Teknik Informatika

Institut Teknologi Bandung

Daftar Isi

1	Teori Singkat	4
1.1	Parser	4
1.1.1	Definisi Parser	4
1.1.2	Peran Parser	4
1.1.3	Keterkaitan Parser dengan Lexer	4
1.2	Recursive Descent	5
1.2.1	Konsep Recursive Descent	5
1.2.2	Cara Kerja Recursive Descent	6
1.3	Parse Tree	7
1.3.1	Definisi Parse Tree	7
1.3.2	Struktur Parse Tree	7
1.3.3	Perbedaan Parse Tree dan AST	8
2	Perancangan dan Implementasi	9
2.1	Bahasa dan library yang digunakan	9
2.2	Struktur Program	9
2.3	Alur Kerja Program	9
2.3.1	Proses Lexical Analysis	10
2.3.2	Proses Parsing	10
2.3.3	Pembentukan Parse Tree	11
2.3.4	Error Handling	12
2.4	Fungsi dan Kelas Utama	13
2.5	Implementasi Recursive Descent	13
2.6	Justifikasi Implementasi	14
3	Hasil Implementasi	14
3.1	Hasil Tokenisasi	14
3.1.1	Program Sederhana	15
3.1.2	Program Kompleks	16
3.1.3	Program Error	17
3.2	Hasil Parse Tree	19
3.2.1	Parse Tree Program Sederhana	19
3.2.2	Parse Tree Program Kompleks	20

3.2.3	Parse Tree Error Handling	22
3.3	Output File	23
3.4	Dokumentasi Program	23
4	Pengujian	24
4.1	Metodologi Pengujian	24
4.2	Test Case 1	25
4.2.1	Tujuan Pengujian	25
4.2.2	Input Program	25
4.2.3	Output Program	25
4.2.4	Hasil Pengujian	26
4.3	Test Case 2	26
4.3.1	Tujuan Pengujian	26
4.3.2	Input Program	26
4.3.3	Output Program	27
4.3.4	Hasil Pengujian	27
4.4	Test Case 3	28
4.4.1	Tujuan Pengujian	28
4.4.2	Input Program	28
4.4.3	Output Program	29
4.4.4	Hasil Pengujian	29
4.5	Test Case 4	30
4.5.1	Tujuan Pengujian	30
4.5.2	Input Program	30
4.5.3	Output Program	31
4.5.4	Hasil Pengujian	31
4.6	Test Case 5	32
4.6.1	Tujuan Pengujian	32
4.6.2	Input Program	32
4.6.3	Output Program	33
4.6.4	Hasil Pengujian	33
5	Aturan Grammar	34
5.1	Grammar Program	34
5.2	Grammar Declaration	34

5.3	Grammar Statement	36
5.4	Grammar Expression	38
5.5	Grammar Lengkap Bahasa Arion	39
6	Kesimpulan dan Saran	45
6.1	Kesimpulan	45
6.2	Saran	46
A	Lampiran	47
A.1	Pranala Ke Repositori	47
A.2	Workspace Diagram Transisi DFA	47
A.3	Pembagian Tugas	47
	Referensi	48

1 Teori Singkat

1.1 Parser

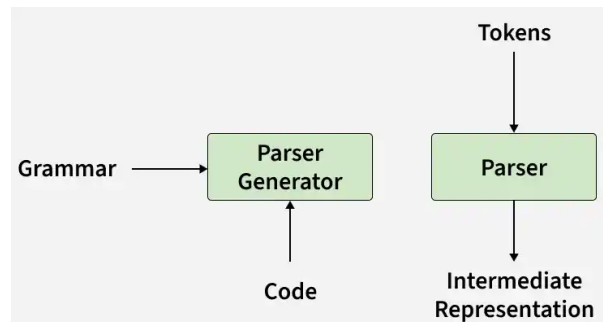


Figure 1: Parsing [3]

1.1.1 Definisi Parser

Parser merupakan komponen pada compiler yang bertugas melakukan analisis sintaks terhadap token hasil proses lexical analysis. Parser memeriksa apakah urutan token yang diterima telah sesuai dengan aturan grammar bahasa pemrograman yang digunakan.

Masukan parser berupa daftar token yang dihasilkan lexer, sedangkan keluarannya berupa representasi struktur sintaks program seperti Parse Tree. Selain melakukan validasi sintaks, parser juga bertanggung jawab mendeteksi kesalahan sintaks (*syntax error*) pada source code.

1.1.2 Peran Parser

Parser memiliki beberapa peran utama dalam proses kompilasi, yaitu:

1. Memastikan urutan token sesuai dengan grammar bahasa Arion.
2. Mendeteksi kesalahan sintaks dan memberikan pesan error yang informatif.
3. Membentuk struktur hierarkis program dalam bentuk Parse Tree.
4. Menjadi penghubung antara tahap lexical analysis dan tahap semantic analysis.

Pada milestone ini, parser digunakan untuk memvalidasi struktur sintaks program Arion berdasarkan grammar yang telah ditentukan menggunakan metode Recursive Descent Parser.

1.1.3 Keterkaitan Parser dengan Lexer

Lexer dan parser merupakan dua tahap yang saling berkaitan dalam proses compiler.

Lexer bertugas membaca source code karakter demi karakter kemudian mengubahnya menjadi token-token. Token tersebut kemudian digunakan parser untuk dianalisis struktur sintaksnya.

Sebagai contoh, source code berikut:

```
1 a := b + 10;
```

akan diubah lexer menjadi token:

```
1 ident(a)
2 becomes
3 ident(b)
4 plus
5 intcon(10)
6 semicolon
```

Parser kemudian memeriksa apakah susunan token tersebut sesuai dengan grammar assignment statement dan expression pada bahasa Arion.

Dengan demikian, lexer berfokus pada analisis leksikal, sedangkan parser berfokus pada analisis sintaks.

1.2 Recursive Descent

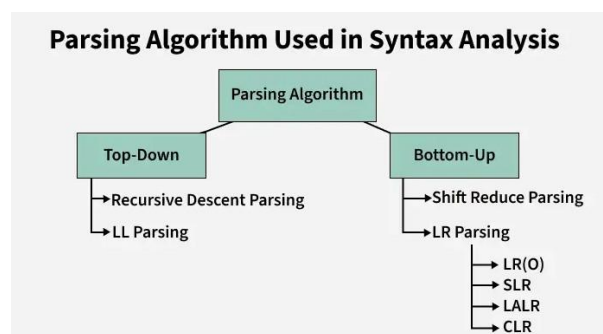


Figure 2: Syntax Analysis [4]

1.2.1 Konsep Recursive Descent

Recursive Descent merupakan metode parsing top-down yang menggunakan sekumpulan fungsi rekursif untuk memproses grammar.

Setiap non-terminal pada grammar direpresentasikan sebagai sebuah fungsi parser. Fungsi tersebut akan memeriksa token masukan berdasarkan aturan produksi yang dimiliki non-terminal terkait.

Sebagai contoh, aturan grammar:

```
1 expression -> term ((plus | minus) term)*
```

dapat direpresentasikan sebagai fungsi:

```
1 parseExpression()
```

yang akan memanggil fungsi `parseTerm()` dan memeriksa keberadaan operator penjumlahan atau pengurangan.

Metode Recursive Descent dipilih karena:

1. Implementasinya sederhana dan mudah dipahami.
2. Struktur program menjadi modular.
3. Cocok digunakan untuk grammar bahasa Arion pada milestone ini.
4. Memudahkan proses debugging dan error handling.

1.2.2 Cara Kerja Recursive Descent

Recursive Descent bekerja dengan membaca token secara berurutan menggunakan pointer token saat ini (*current token*).

Alur kerja parser secara umum adalah sebagai berikut:

1. Parser memulai parsing dari non-terminal root, yaitu **program**.
2. Setiap fungsi parser memeriksa token yang sesuai dengan aturan grammar.
3. Jika token sesuai, parser akan berpindah ke token berikutnya.
4. Jika ditemukan non-terminal lain, parser memanggil fungsi parser terkait secara rekursif.
5. Jika token tidak sesuai dengan grammar, parser menghasilkan syntax error.
6. Jika seluruh token berhasil diproses, maka source code dinyatakan valid.

Sebagai contoh:

```
1 program Hello;  
2 begin  
3 end.
```

Parser akan melakukan proses berikut:

1. Memanggil `parseProgram()`.
2. Memeriksa token `program`sy.
3. Memeriksa `ident`.
4. Memeriksa `semicolon`.
5. Memanggil `parseCompoundStatement()`.
6. Memeriksa token `beginsy`, `endsy`, dan `period`.

Jika seluruh proses berhasil dilakukan tanpa error, maka program valid secara sintaks.

1.3 Parse Tree

1.3.1 Definisi Parse Tree

Parse Tree merupakan representasi struktur sintaks program berdasarkan grammar bahasa yang digunakan.

Setiap node pada Parse Tree merepresentasikan simbol terminal maupun non-terminal pada grammar. Struktur pohon menunjukkan hubungan hierarkis antar aturan produksi yang digunakan selama proses parsing.

Pada milestone ini, Parse Tree dihasilkan sebagai output utama parser dan digunakan untuk merepresentasikan struktur program Arion secara lengkap.

1.3.2 Struktur Parse Tree

Parse Tree terdiri dari root node, internal node, dan leaf node.

1. Root node merepresentasikan non-terminal utama, yaitu `<program>`.
2. Internal node merepresentasikan non-terminal lain seperti `<expression>` atau `<statement>`.
3. Leaf node merepresentasikan token terminal hasil lexical analysis.

Sebagai contoh, source code:

```
1 a := 5;
```

dapat direpresentasikan dalam Parse Tree sederhana sebagai:

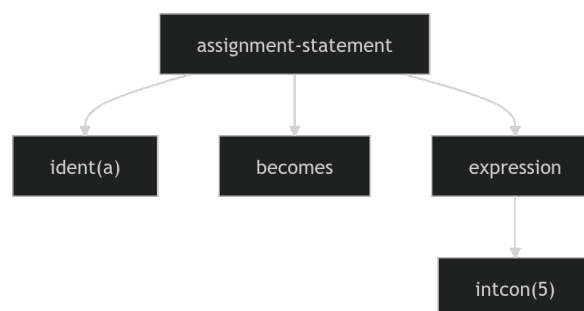


Figure 3: Ex. Parse Tree

Struktur pohon tersebut menunjukkan bahwa statement assignment terdiri dari identifier, operator assignment, dan expression.

Parse Tree membantu visualisasi bagaimana grammar digunakan parser untuk membentuk struktur program secara hierarkis.

1.3.3 Perbedaan Parse Tree dan AST

Parse Tree dan Abstract Syntax Tree (AST) sama-sama digunakan untuk merepresentasikan struktur program, namun keduanya memiliki tujuan dan tingkat detail yang berbeda.

Parse Tree merepresentasikan seluruh aturan grammar yang digunakan selama proses parsing. Semua simbol terminal dan non-terminal ditampilkan secara lengkap sesuai struktur grammar bahasa. Oleh karena itu, Parse Tree cenderung memiliki struktur yang lebih besar dan detail.

Sebaliknya, AST merupakan representasi sintaks yang lebih sederhana dengan menghilangkan node-node yang tidak penting terhadap makna program. AST hanya mempertahankan struktur inti yang dibutuhkan pada tahap semantic analysis maupun code generation.

Sebagai contoh, statement:

```
1 a := b + 5;
```

pada Parse Tree akan memuat node seperti:

```
1 <assignment-statement>
2 <expression>
3 <simple-expression>
4 <term>
5 <factor>
```

sedangkan pada AST hanya direpresentasikan sebagai operasi assignment dan operasi penjumlahan tanpa node grammar tambahan.

Perbedaan utama Parse Tree dan AST dapat dilihat pada Tabel berikut.

Aspek	Parse Tree	AST
Representasi grammar	Lengkap sesuai grammar	Disederhanakan
Node non-terminal	Ditampilkan semua	Hanya yang penting
Ukuran struktur	Lebih besar dan detail	Lebih ringkas
Tujuan utama	Validasi sintaks	Semantic analysis dan code generation
Kesesuaian dengan grammar	Sangat bergantung pada grammar	Lebih abstrak

Table 1: Perbedaan Parse Tree dan AST

Pada milestone ini, struktur yang dihasilkan parser adalah Parse Tree dan bukan AST karena fokus utama tahap syntax analysis adalah memvalidasi struktur grammar program secara lengkap.

2 Perancangan dan Implementasi

2.1 Bahasa dan library yang digunakan

Program dikembangkan menggunakan bahasa pemrograman **C++** karena memiliki performa yang baik serta mendukung implementasi struktur data dan rekursi yang dibutuhkan dalam parser Recursive Descent.

Library utama yang digunakan antara lain:

1. **<vector>** Digunakan untuk menyimpan daftar token dan child node pada Parse Tree.
2. **<memory>** Digunakan untuk `std::shared_ptr` pada node Parse Tree agar pengelolaan memori dilakukan secara otomatis.
3. **<iostream>** Digunakan untuk proses input dan output program.
4. **<string>** Digunakan untuk manipulasi string source code dan token.
5. **Recursive Function** Digunakan untuk implementasi parser Recursive Descent dan pencetakan Parse Tree.

2.2 Struktur Program

Program dibagi menjadi beberapa modul utama agar implementasi lebih modular dan mudah dikembangkan.

1. **Lexer** Bertugas melakukan lexical analysis dan menghasilkan token dari source code.
2. **Parser** Bertugas melakukan syntax analysis menggunakan metode Recursive Descent.
3. **Parse Tree** Digunakan untuk merepresentasikan struktur sintaks program dalam bentuk tree.
4. **Utility** Berisi fungsi tambahan seperti konversi token menjadi string dan helper lainnya.

Setiap komponen dipisahkan ke file yang berbeda sehingga alur program lebih mudah dipahami dan proses debugging menjadi lebih mudah.

2.3 Alur Kerja Program

Program bekerja melalui beberapa tahap utama mulai dari pembacaan source code hingga pembentukan Parse Tree.

2.3.1 Proses Lexical Analysis

Tahap lexical analysis dilakukan oleh class `Lexer`. Lexer membaca source code karakter demi karakter menggunakan DFA (*Deterministic Finite Automata*) untuk menentukan token yang sesuai.

Lexer menyimpan state saat ini menggunakan variabel:

```
1 State state = STATE_START;
```

Transisi state dilakukan menggunakan fungsi:

```
1 DFA::getNextState(state, c)
```

Jika state yang dicapai merupakan accepting state, lexer akan menghasilkan token menggunakan fungsi:

```
1 emitToken(...)
```

Lexer mendukung beberapa jenis token seperti:

1. Keyword
2. Identifier
3. Integer constant
4. Real constant
5. String constant
6. Character constant
7. Operator
8. Separator
9. Comment

Selain itu lexer juga mendeteksi token tidak valid dan menghasilkan token:

```
1 TOKEN_UNKNOWN
```

untuk keperluan error handling.

2.3.2 Proses Parsing

Tahap parsing dilakukan menggunakan metode Recursive Descent Parser.

Parser membaca token hasil lexer menggunakan pointer global:

```
1 static size_t g_pos;
```

Token saat ini diperoleh menggunakan fungsi:

```
1 current()
```

Sedangkan pengecekan token dilakukan menggunakan:

```
1 check(TokenType t)
```

Setiap aturan grammar direpresentasikan sebagai fungsi parser tersendiri, contohnya:

```
1 parseProgram()
2 parseStatement()
3 parseExpression()
4 parseFactor()
```

Parser dimulai dari fungsi:

```
1 parse()
```

yang kemudian memanggil non-terminal root:

```
1 parseProgram()
```

Token yang sesuai grammar akan dikonsumsi menggunakan fungsi:

```
1 consume()
```

sedangkan validasi token dilakukan menggunakan:

```
1 expect(TokenType t)
```

2.3.3 Pembentukan Parse Tree

Parse Tree dibangun secara langsung selama proses parsing berlangsung.

Setiap node direpresentasikan menggunakan class `ParseNode` yang memiliki:

1. Label node
2. Daftar child node

Node dibuat menggunakan:

```
1 std::make_shared<ParseNode>()
```

dan child node ditambahkan menggunakan:

```
1 node->children.push_back(...)
```

Token terminal diubah menjadi node menggunakan fungsi:

```
1 makeTerminal(...)
```

Setelah proses parsing selesai, Parse Tree dicetak menggunakan fungsi:

```
1 printTree(...)
```

Pencetakan tree dilakukan secara rekursif menggunakan helper:

```
1 printTreeHelper(...)
```

Struktur tree ditampilkan menggunakan karakter ASCII:

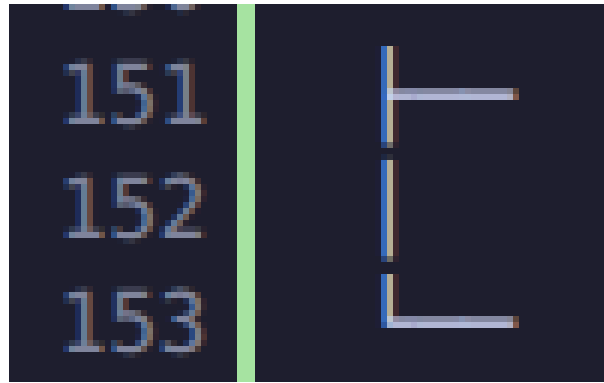


Figure 4: ASCII for Tree

sehingga hubungan antar node lebih mudah dibaca.

Apabila tree kosong, program akan menampilkan:

```
1 (empty tree)
```

2.3.4 Error Handling

Program memiliki mekanisme error handling pada lexer maupun parser.

Pada lexer, token yang tidak valid akan menghasilkan pesan error seperti:

```
1 Lexer error (line x, col y):
2 token tidak valid
```

Lexer juga menyimpan posisi line dan column agar lokasi error dapat diketahui secara spesifik.

Pada parser, error ditangani menggunakan fungsi:

```
1 syntaxError(...)
```

Pesan error parser memuat:

1. Line error
2. Column error
3. Token yang ditemukan
4. Token yang diharapkan

Contoh:

```
1 Syntax error (line 5, col 10):
2 unexpected token 'period',
3 expected semicolon
```

Parser juga membuat node error menggunakan:

```
1 makeErrorNode(...)
```

dan:

```
1 makeUnexpectedNode()
```

sehingga Parse Tree tetap dapat dibentuk meskipun terjadi syntax error.

Status error parser disimpan menggunakan:

```
1 g_errorFlag
```

2.4 Fungsi dan Kelas Utama

Fungsi/Kelas	Deskripsi
Lexer	Melakukan lexical analysis
scan()	Membaca source code dan menghasilkan token
emitToken()	Membentuk token hasil scanning
parse()	Entry point parser
parseProgram()	Memproses grammar program
parseStatement()	Memproses statement
parseExpression()	Memproses expression
parseVariable()	Memproses variable dan component variable
parseFactor()	Memproses factor pada expression
expect()	Validasi token sesuai grammar
consume()	Mengonsumsi token saat ini
ParseNode	Representasi node Parse Tree
printTree()	Menampilkan Parse Tree
printTreeHelper()	Helper rekursif pencetakan tree

Table 2: Fungsi dan Kelas Utama

2.5 Implementasi Recursive Descent

Parser diimplementasikan menggunakan pendekatan top-down Recursive Descent.

Setiap non-terminal grammar direpresentasikan sebagai fungsi rekursif. Sebagai contoh:

```
1 <expression> ->
2   <simple-expression>
3   ((relational-operator)
4   <simple-expression>)?
```

diimplementasikan menggunakan:

```
1 parseExpression()
```

Fungsi parser saling memanggil sesuai struktur grammar sehingga membentuk proses parsing yang hierarkis.

Sebagai contoh:

1. `parseProgram()` memanggil `parseCompoundStatement()`

2. `parseCompoundStatement()` memanggil `parseStatementList()`
3. `parseStatementList()` memanggil `parseStatement()`
4. `parseStatement()` dapat memanggil `parseExpression()`

Implementasi parser juga mendukung:

1. Assignment statement
2. Conditional statement
3. Loop statement
4. Procedure/function call
5. Array indexing
6. Record field access

2.6 Justifikasi Implementasi

Metode Recursive Descent dipilih karena sesuai dengan grammar bahasa Arion pada milestone ini.

Beberapa alasan pemilihan implementasi adalah:

1. Struktur parser lebih mudah dipahami karena setiap grammar direpresentasikan sebagai fungsi.
2. Implementasi lebih modular dan mudah dikembangkan.
3. Error handling dapat dikontrol secara manual.
4. Parse Tree dapat dibangun langsung selama proses parsing.
5. Pendekatan rekursif cocok untuk struktur grammar yang hierarkis.
6. Penggunaan `shared_ptr` mempermudah pengelolaan memori node Parse Tree secara otomatis.

Selain itu, implementasi Parse Tree secara rekursif mempermudah visualisasi struktur sintaks program dalam bentuk tree yang terformat dengan jelas.

3 Hasil Implementasi

3.1 Hasil Tokenisasi

Pada tahap lexical analysis, source code Arion diubah menjadi token-token yang kemudian digunakan pada tahap parsing.

3.1.1 Program Sederhana

Berikut contoh source code sederhana yang digunakan pada pengujian:

```
1 program CekKelulusan;  
2 var  
3   nilaiTubes, nilaiUjian: integer;  
4   status: string;  
5 begin  
6   nilaiTubes := 75;  
7   nilaiUjian := 80;  
8  
9   if (nilaiTubes >= 70) and (nilaiUjian <> 0) then  
10    status := 'Lulus'  
11  else  
12    status := 'Mengulang';  
13 end.
```

Hasil tokenisasi program:

```
1 programsy  
2 ident (CekKelulusan)  
3 semicolon  
4 varsy  
5 ident (nilaiTubes)  
6 comma  
7 ident (nilaiUjian)  
8 colon  
9 ident (integer)  
10 semicolon  
11 ident (status)  
12 colon  
13 ident (string)  
14 semicolon  
15 beginsy  
16 ident (nilaiTubes)  
17 becomes  
18 intcon (75)  
19 semicolon  
20 ident (nilaiUjian)  
21 becomes  
22 intcon (80)  
23 semicolon  
24 ifsy  
25 lparent  
26 ident (nilaiTubes)  
27 geq  
28 intcon (70)  
29 rparent  
30 andsy  
31 lparent  
32 ident (nilaiUjian)  
33 neq  
34 intcon (0)  
35 rparent  
36 thensy  
37 ident (status)  
38 becomes  
39 string ('Lulus')
```

```
40 elsesy
41  ident (status)
42  becomes
43  string ('Mengulang')
44  semicolon
45  endsy
46  period
```

3.1.2 Program Kompleks

Program kompleks digunakan untuk menguji fitur parser yang lebih lengkap seperti:

1. Enumerated type
2. Record type
3. Nested array
4. Procedure
5. Procedure call
6. Array indexing
7. Record field access
8. Expression
9. Compound statement

Berikut contoh source code kompleks yang digunakan pada pengujian:

```
1  program Milestone2Valid;
2
3  type
4      Warna == (Merah, Kuning, Hijau);
5      Kendaraan == record
6          jenis : string;
7          tahun : integer;
8      end;
9
10 var
11     pi : real;
12     matriks : array [1..5] of array [1..5] of integer;
13     titik : Kendaraan;
14     skala : array [0..10] of real;
15 procedure ResetData;
16 begin
17     pi := 3.14; { Realcon }
18 end;
19
20
21 begin
22     matriks[1][2] := 10;
23     titik.jenis := 'Mobil';
24     ResetData;
```

```

25   ResetData();
26   Tampilkan(pi);
27   if (pi > 3.0) then
28   begin
29       pi := pi * 1.5;
30   end;
31 end.

```

Contoh hasil tokenisasi:

```

1  programsy
2  ident (Milestone2Valid)
3  semicolon
4  typesy
5  ident (Warna)
6  eql
7  lparent
8  ident (Merah)
9  comma
10 ident (Kuning)
11 comma
12 ident (Hijau)
13 rparent
14 semicolon
15 ...

```

Hasil tokenisasi menunjukkan bahwa lexer berhasil mengenali berbagai jenis token seperti keyword, identifier, operator, constant, separator, dan comment.

3.1.3 Program Error

Selain program valid, pengujian juga dilakukan menggunakan source code yang mengandung berbagai syntax error untuk menguji mekanisme error handling pada lexer dan parser.

Berikut contoh source code error yang digunakan:

```

1  program ErrorHandlerTest;
2
3  const
4      pi == 3.14
5
6  type
7      Warna == (Merah, Kuning Hijau);
8
9      Data == record
10         nama : string
11         umur : integer;
12     end;
13
14  var
15      angka : array [1..10] of integer;
16      matriks : array [1..5] of array [1..5] of integer;
17      x : integer;
18      y : real;
19
20  procedure Cetak(a : integer; b : real

```

```
21 begin
22     x := 10
23 end;
24
25 begin
26
27     @invalidToken;
28
29     x := * 5;
30
31     y := (3.14 + );
32
33     matriks[1][ := 10;
34
35     if (x > ) then
36     begin
37         y := y * 2;
38     end;
39
40     if x > 0
41     begin
42         x := x + 1;
43     end;
44
45     Cetak(1, , 2);
46
47     angka[1 := 100;
48
49     Data.nama := 'Budi';
50
51     x := 10 + * 20;
52
53     while x < do
54     begin
55         x := x + 1;
56     end;
57
58     repeat
59         x := x - 1
60     until ;
61
62     for x := 1 10 do
63     begin
64         y := y + 1.0;
65     end;
66
67     case x of
68         1:
69             y := 1.0;
70         :
71             y := 2.0;
72     end;
73
74 end.
```

Hasil tokenisasi menunjukkan bahwa lexer berhasil mendeteksi token tidak valid:

```
1 UNKNOWN (@invalidToken)
```

Selain itu, parser juga berhasil mendeteksi berbagai syntax error seperti:

1. Missing semicolon
2. Missing rparent
3. Invalid expression
4. Unexpected token

Contoh error node yang dihasilkan parser:

```
1 ERROR(unexpected:typesy , expected:semicolon)
1 ERROR(unexpected:ident (Hijau), expected:rparent)
```

3.2 Hasil Parse Tree

Setelah proses tokenisasi selesai, parser melakukan syntax analysis dan membentuk Parse Tree berdasarkan grammar bahasa Arion.

3.2.1 Parse Tree Program Sederhana

Berikut sebagian hasil Parse Tree dari program sederhana:

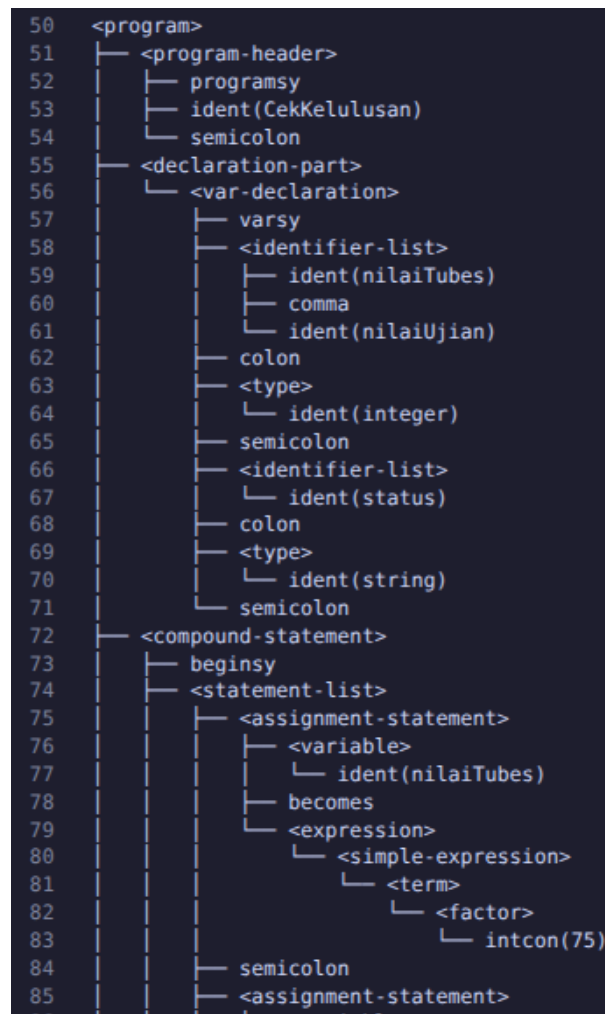


Figure 5: Hasil parse tree program sederhana

Parse Tree menunjukkan struktur hierarkis program berdasarkan grammar yang digunakan parser.

3.2.2 Parse Tree Program Kompleks

Program kompleks menghasilkan Parse Tree yang lebih besar karena melibatkan banyak aturan grammar seperti:

1. Type declaration
2. Record type
3. Array type
4. Procedure declaration
5. Nested compound statement
6. Variable indexing

7. Procedure/function call

Berikut sebagian hasil Parse Tree:

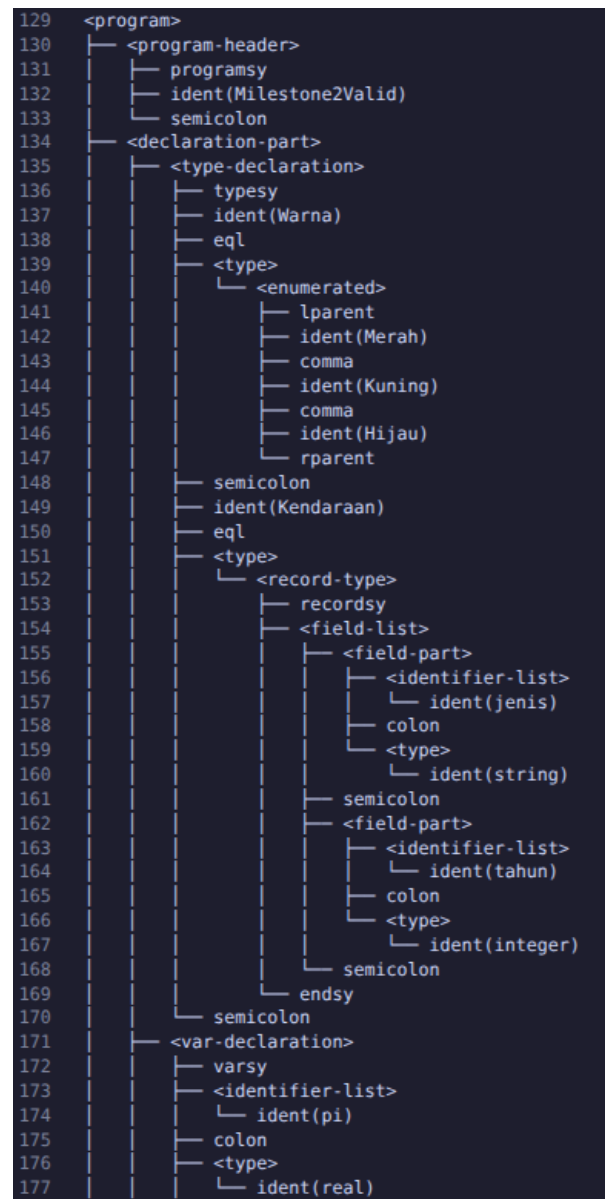


Figure 6: Hasil parse tree program kompleks

Selain itu parser juga berhasil membentuk Parse Tree untuk:

1. Array multidimensi
2. Record field access
3. Nested expression
4. Procedure call dengan parameter

3.2.3 Parse Tree Error Handling

Program juga diuji menggunakan source code yang mengandung berbagai syntax error untuk menguji robustness parser.

Beberapa error yang diuji antara lain:

1. Missing semicolon
2. Kurung tidak tertutup
3. Invalid expression
4. Missing operator
5. Invalid array indexing
6. Unexpected token

Berikut hasil Parse Tree:

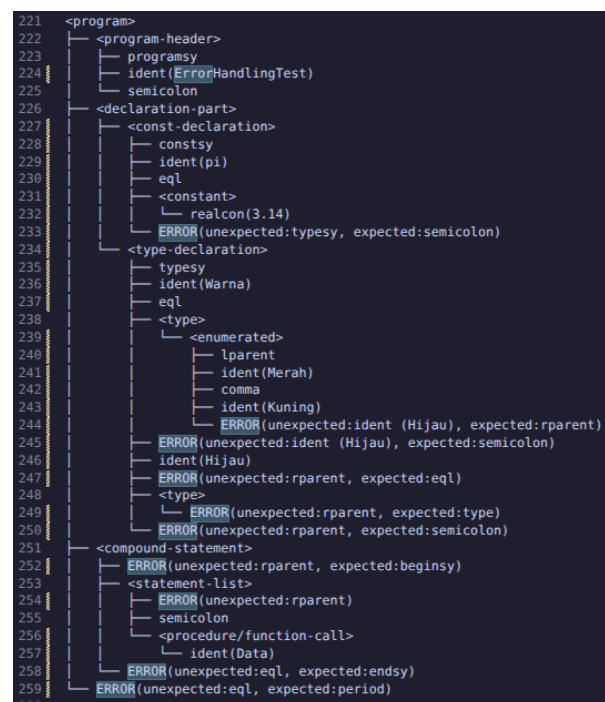


Figure 7: Hasil parse tree program error

Contoh error token:

```
1 UNKNOWN (@invalidToken)
```

Parser menghasilkan error node seperti:

```
1 ERROR(unexpected:typesy, expected:semicolon)
```

dan:

```
1 ERROR(unexpected:ident (Hijau), expected:rparent)
```

Hal ini menunjukkan bahwa parser mampu:

1. Memberikan pesan error yang informatif
2. Menampilkan token yang ditemukan
3. Menampilkan token yang diharapkan
4. Tetap membentuk Parse Tree parsial meskipun terjadi error

3.3 Output File

Program menghasilkan output Parse Tree baik ke terminal maupun file text.

File output digunakan untuk:

1. Menyimpan hasil tokenisasi
2. Menyimpan hasil Parse Tree
3. Mempermudah proses debugging
4. Mempermudah dokumentasi pengujian

Isi file pada **milestone-1** berisi hasil lexical analysis, sedangkan file pada **milestone-2** berisi Parse Tree hasil parsing.

3.4 Dokumentasi Program

Program berhasil mengimplementasikan parser Recursive Descent untuk bahasa Arion sesuai grammar yang ditentukan.

Fitur yang berhasil diimplementasikan antara lain:

1. Declaration parsing
2. Variable parsing
3. Array parsing
4. Record parsing
5. Procedure declaration
6. Assignment statement
7. Conditional statement
8. Loop statement
9. Procedure/function call
10. Expression parsing

11. Parse Tree generation
12. Error handling

Selain itu, parser juga mendukung:

1. Array multidimensi
2. Record field access
3. Nested expression
4. Nested compound statement
5. Syntax recovery menggunakan error node

Implementasi Parse Tree menggunakan pendekatan rekursif sehingga struktur sintaks program dapat divisualisasikan dengan jelas dalam bentuk tree.

4 Pengujian

4.1 Metodologi Pengujian

Pengujian dilakukan untuk memastikan parser Recursive Descent dapat:

1. Mengenali struktur sintaks program Arion sesuai grammar.
2. Membentuk Parse Tree yang benar.
3. Menangani berbagai jenis statement dan declaration.
4. Menangani syntax error tanpa menyebabkan program crash.

Setiap pengujian dilakukan dengan langkah berikut:

1. Menjalankan lexer untuk menghasilkan token.
2. Mengirim daftar token ke parser.
3. Membentuk Parse Tree menggunakan Recursive Descent.
4. Memeriksa kesesuaian Parse Tree dengan grammar.
5. Memeriksa error handling apabila source code mengandung syntax error.

Pengujian dilakukan menggunakan beberapa variasi source code yang mencakup:

1. Assignment statement
2. Conditional statement
3. Array dan record
4. Procedure/function call
5. Error handling

4.2 Test Case 1

4.2.1 Tujuan Pengujian

Menguji parsing assignment statement dan expression sederhana.

4.2.2 Input Program

```

1 program AssignmentTest;
2
3 var
4     a : integer;
5
6 begin
7     a := 5 + 10;
8 end.
```

4.2.3 Output Program

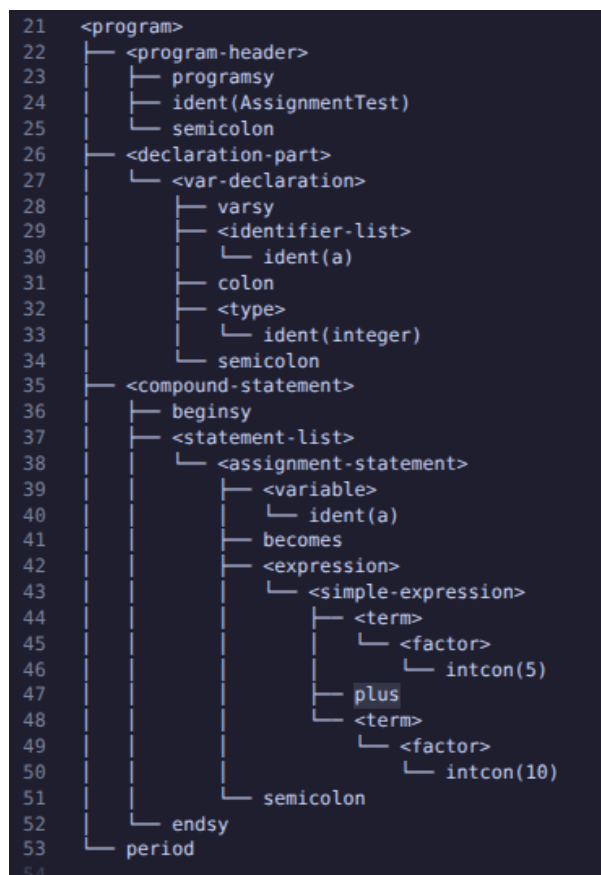


Figure 8: Test Case 1

Selengkapnya lihat file milestone-2/output-1.txt pada repository (lihat Lampiran A.1).

4.2.4 Hasil Pengujian

Parser berhasil:

1. Mengenali declaration.
2. Mengenali assignment statement.
3. Mengenali expression dengan operator penjumlahan.
4. Membentuk Parse Tree sesuai grammar.

Test case berhasil dijalankan tanpa syntax error dan Parse Tree terbentuk dengan benar.

4.3 Test Case 2

4.3.1 Tujuan Pengujian

Menguji parsing conditional statement menggunakan operator relasional dan operator logika.

4.3.2 Input Program

```
1 program CekKelulusan;
2
3 var
4     nilaiTubes, nilaiUjian: integer;
5     status: string;
6
7 begin
8     nilaiTubes := 75;
9     nilaiUjian := 80;
10
11     if (nilaiTubes >= 70) and (nilaiUjian <> 0) then
12         status := 'Lulus'
13     else
14         status := 'Mengulang';
15 end.
```

4.3.3 Output Program

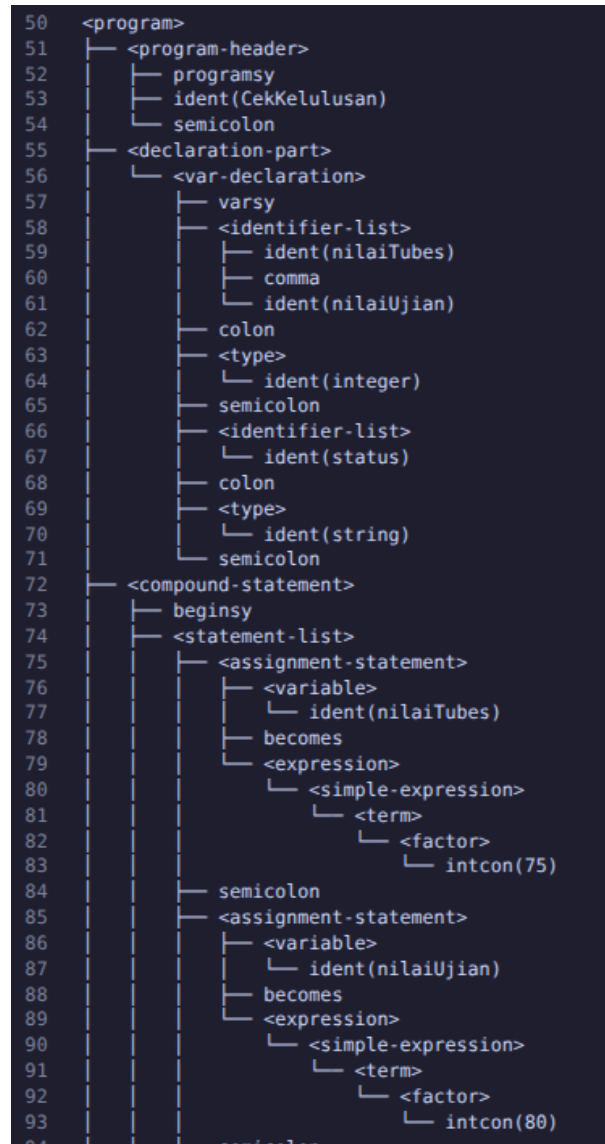


Figure 9: Test Case 2

Selengkapnya lihat file milestone-2/output-2.txt pada repository (lihat Lampiran A.1).

4.3.4 Hasil Pengujian

Parser berhasil:

1. Mengenali if statement.
2. Mengenali operator relasional $\geq$ dan $<>$.
3. Mengenali operator logika and.
4. Mengenali string constant.

5. Membentuk Parse Tree hierarkis untuk expression kompleks.

Conditional statement berhasil diparsing dengan benar dan Parse Tree sesuai grammar.

4.4 Test Case 3

4.4.1 Tujuan Pengujian

Menguji parsing array multidimensi dan record field access.

4.4.2 Input Program

```
1 program ArrayRecordTest;  
2  
3 type  
4     Kendaraan == record  
5         jenis : string;  
6         tahun : integer;  
7     end;  
8  
9 var  
10     matriks : array [1..5] of array [1..5] of integer;  
11     data : Kendaraan;  
12  
13 begin  
14     matriks[1][2] := 10;  
15     data.jenis := 'Mobil';  
16 end.
```

4.4.3 Output Program



Figure 10: Test Case 3

Selengkapnya lihat file *milestone-2/output-3.txt* pada repository (lihat Lampiran A.1).

4.4.4 Hasil Pengujian

Parser berhasil:

1. Mengenali nested array type.
2. Mengenali multidimensional indexing.
3. Mengenali component variable.
4. Mengenali record field access menggunakan operator period.
5. Membentuk Parse Tree untuk variable kompleks.

Array multidimensi dan record field access berhasil diparsing.

4.5 Test Case 4

4.5.1 Tujuan Pengujian

Menguji parsing procedure declaration dan procedure/function call.

4.5.2 Input Program

```
1 program ProcedureTest;  
2  
3 procedure Cetak(x : integer; y : real);  
4 begin  
5 end;  
6  
7 begin  
8     Cetak(10, 3.14);  
9 end.
```

4.5.3 Output Program

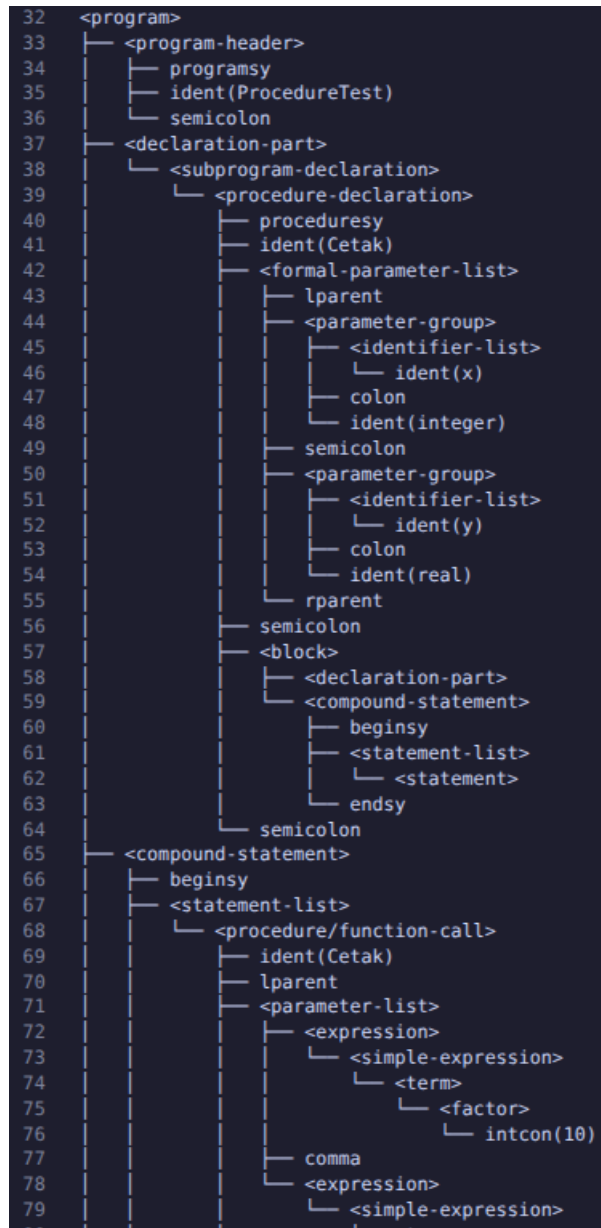


Figure 11: Test Case 4

Selengkapnya lihat file *milestone-2/output-4.txt* pada repository (lihat Lampiran A.1).

4.5.4 Hasil Pengujian

Parser berhasil:

1. Mengenali procedure declaration.
2. Mengenali formal parameter list.
3. Mengenali parameter list pada procedure call.

4. Mengenali real constant.
5. Membentuk Parse Tree untuk procedure/function-call.

Procedure declaration dan procedure call berhasil diparsing dengan benar.

4.6 Test Case 5

4.6.1 Tujuan Pengujian

Menguji mekanisme syntax error handling dan recovery parser.

4.6.2 Input Program

```
1 program ErrorHandlingTest;  
2  
3 const  
4     pi == 3.14  
5  
6 type  
7     Warna == (Merah, Kuning Hijau);  
8  
9 begin  
10     @invalidToken;  
11  
12     x := * 5;  
13 end.
```

4.6.3 Output Program

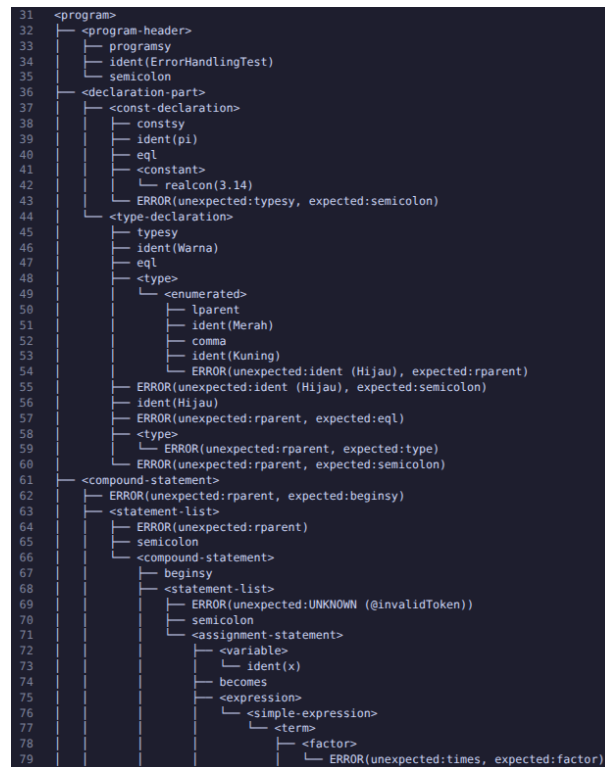


Figure 12: Test Case 5

Selengkapnya lihat file *milestone-2/output-5.txt* pada repository (lihat Lampiran A.1).

4.6.4 Hasil Pengujian

Lexer dan parser berhasil mendeteksi berbagai syntax error seperti:

1. Missing semicolon
2. Missing separator
3. Invalid expression
4. Unexpected token
5. Unknown token

Contoh error yang dihasilkan:

```
1 UNKNOWN (@invalidToken)
1 ERROR(unexpected:typesy , expected:semicolon)
```

Parser juga tetap menghasilkan Parse Tree parsial meskipun source code mengandung error.

Mekanisme error handling berhasil berjalan dengan baik dan parser tidak mengalami crash saat menemukan syntax error.

5 Aturan Grammar

Grammar pada parser diimplementasikan menggunakan pendekatan Recursive Descent. Setiap non-terminal direpresentasikan sebagai fungsi parser pada program.

5.1 Grammar Program

```
<program> ->
    <program-header>
    <declaration-part>
    <compound-statement>
    period

<program-header> ->
    programsy ident semicolon

<block> ->
    <declaration-part>
    <compound-statement>
```

5.2 Grammar Declaration

```
<declaration-part> ->
    (<const-declaration>)*
    (<type-declaration>)*
    (<var-declaration>)*
    (<subprogram-declaration>)*

<const-declaration> ->
    constsy
    (ident eql <constant> semicolon)+

<constant> ->
    charcon
    | string
    | ((plus | minus)? (ident | intcon | realcon))

<type-declaration> ->
    typesy
    (ident eql <type> semicolon)+

<var-declaration> ->
    varsy
    (<identifier-list> colon <type> semicolon)+

<identifier-list> ->
    ident
```

```
(comma ident)*

<type> ->
  ident
  | <array-type>
  | <range>
  | <enumerated>
  | <record-type>

<array-type> ->
  arraysy
  lbrack
  (<range> | ident)
  rbrack
  ofsy
  <type>

<range> ->
  <expression>
  period
  period
  <expression>

<enumerated> ->
  lparent
  ident
  (comma ident)*
  rparent

<record-type> ->
  recordsy
  <field-list>
  endsy

<field-list> ->
  <field-part>
  (semicolon <field-part>)*

<field-part> ->
  <identifier-list>
  colon
  <type>

<subprogram-declaration> ->
  <procedure-declaration>
  | <function-declaration>

<procedure-declaration> ->
```

```
proceduresy
ident
(<formal-parameter-list>)?
semicolon
<block>
semicolon

<function-declaration> ->
  functionsy
  ident
  (<formal-parameter-list>)?
  colon
  ident
  semicolon
  <block>
  semicolon

<formal-parameter-list> ->
  lparent
  <parameter-group>
  (semicolon <parameter-group>)*
  rparent

<parameter-group> ->
  <identifier-list>
  colon
  (ident | <array-type>)
```

5.3 Grammar Statement

```
<compound-statement> ->
  beginsy
  <statement-list>
  endsy

<statement-list> ->
  <statement>
  (semicolon <statement>)*

<statement> ->
  <assignment-statement>
  | <if-statement>
  | <case-statement>
  | <while-statement>
  | <repeat-statement>
  | <for-statement>
  | <procedure/function-call>
```

```
| <compound-statement>
| empty

<variable> ->
  ident
  (<component-variable>)*

<component-variable> ->
  (lbrack <index-list> rbrack)
  | (period ident)

<index-list> ->
  (intcon | charcon | ident)
  (comma (intcon | charcon | ident))*

<assignment-statement> ->
  <variable>
  becomes
  <expression>

<if-statement> ->
  ifsy
  <expression>
  thensy
  <statement>
  (elsesy <statement>)?

<case-statement> ->
  casesy
  <expression>
  ofsy
  <case-block>
  endsy

<case-block> ->
  <constant>
  (comma <constant>)*
  colon
  <statement>
  (semicolon <case-block>?)*

<while-statement> ->
  whilesy
  <expression>
  dosy
  <statement>

<repeat-statement> ->
```

```
    repeatsy
    <statement-list>
    untilsy
    <expression>

<for-statement> ->
    forsy
    ident
    becomes
    <expression>
    (tosy | downtosy)
    <expression>
    dosy
    <statement>

<procedure/function-call> ->
    ident
    (
        lparent
        (<parameter-list>)?
        rparent
    )?

<parameter-list> ->
    <expression>
    (comma <expression>)*
```

5.4 Grammar Expression

```
<expression> ->
    <simple-expression>
    (
        <relational-operator>
        <simple-expression>
    )?

<simple-expression> ->
    (plus | minus)?
    <term>
    (
        <additive-operator>
        <term>
    )*

<term> ->
    <factor>
    (
```

```
        <multiplicative-operator>
        <factor>
    )*

<factor> ->
    ident
    | intcon
    | realcon
    | charcon
    | string
    | (lparent <expression> rparent)
    | (notsy <factor>)
    | <procedure/function-call>
    | <variable>

<relational-operator> ->
    eql
    | neq
    | gtr
    | geq
    | lss
    | leq

<additive-operator> ->
    plus
    | minus
    | orsy

<multiplicative-operator> ->
    times
    | rdiv
    | idiv
    | imod
    | andsy
```

5.5 Grammar Lengkap Bahasa Arion

```
<program> ->
    <program-header>
    <declaration-part>
    <compound-statement>
    period

<program-header> ->
    programsy ident semicolon

<declaration-part> ->
```

```
(<const-declaration>)*
(<type-declaration>)*
(<var-declaration>)*
(<subprogram-declaration>)*

<const-declaration> ->
  constsy
  (ident eql <constant> semicolon)+

<constant> ->
  charcon
  | string
  | ((plus | minus)? (ident | intcon | realcon))

<type-declaration> ->
  typesy
  (ident eql <type> semicolon)+

<var-declaration> ->
  varsy
  (<identifier-list> colon <type> semicolon)+

<identifier-list> ->
  ident
  (comma ident)*

<type> ->
  ident
  | <array-type>
  | <range>
  | <enumerated>
  | <record-type>

<array-type> ->
  arraysy
  lbrack
  (<range> | ident)
  rbrack
  ofsy
  <type>

<range> ->
  <expression>
  period
  period
  <expression>

<enumerated> ->
```

```
lparent
ident
(commma ident)*
rparent

<record-type> ->
    recordsy
    <field-list>
    endsy

<field-list> ->
    <field-part>
    (semicolon <field-part>)*

<field-part> ->
    <identifier-list>
    colon
    <type>

<subprogram-declaration> ->
    <procedure-declaration>
    | <function-declaration>

<procedure-declaration> ->
    proceduresy
    ident
    (<formal-parameter-list>)?
    semicolon
    <block>
    semicolon

<function-declaration> ->
    functionsy
    ident
    (<formal-parameter-list>)?
    colon
    ident
    semicolon
    <block>
    semicolon

<block> ->
    <declaration-part>
    <compound-statement>

<formal-parameter-list> ->
    lparent
    <parameter-group>
```

```
(semicolon <parameter-group>)*
rparent

<parameter-group> ->
  <identifier-list>
  colon
  (ident | <array-type>)

<compound-statement> ->
  beginsy
  <statement-list>
  endsy

<statement-list> ->
  <statement>
  (semicolon <statement>)*

<statement> ->
  <assignment-statement>
  | <if-statement>
  | <case-statement>
  | <while-statement>
  | <repeat-statement>
  | <for-statement>
  | <procedure/function-call>
  | <compound-statement>
  | empty

<variable> ->
  ident
  (<component-variable>)*

<component-variable> ->
  (lbrack <index-list> rbrack)
  | (period ident)

<index-list> ->
  (intcon | charcon | ident)
  (comma (intcon | charcon | ident))*

<assignment-statement> ->
  <variable>
  becomes
  <expression>

<if-statement> ->
  ifsy
  <expression>
```

```
    thensy
    <statement>
    (elsesy <statement>)?

<case-statement> ->
    casesy
    <expression>
    ofsy
    <case-block>
    endsy

<case-block> ->
    <constant>
    (comma <constant>)*
    colon
    <statement>
    (semicolon <case-block>?)*

<while-statement> ->
    whilesy
    <expression>
    dosy
    <statement>

<repeat-statement> ->
    repeatsy
    <statement-list>
    untilsy
    <expression>

<for-statement> ->
    forsy
    ident
    becomes
    <expression>
    (tosy | downtosy)
    <expression>
    dosy
    <statement>

<procedure/function-call> ->
    ident
    (
        lparent
        (<parameter-list>)?
        rparent
    )?
```

```
<parameter-list> ->
    <expression>
    (comma <expression>)*

<expression> ->
    <simple-expression>
    (
        <relational-operator>
        <simple-expression>
    )?

<simple-expression> ->
    (plus | minus)?
    <term>
    (
        <additive-operator>
        <term>
    )*

<term> ->
    <factor>
    (
        <multiplicative-operator>
        <factor>
    )*

<factor> ->
    ident
    | intcon
    | realcon
    | charcon
    | string
    | (lparent <expression> rparent)
    | (notsy <factor>)
    | <procedure/function-call>
    | <variable>

<relational-operator> ->
    eql
    | neq
    | gtr
    | geq
    | lss
    | leq

<additive-operator> ->
    plus
    | minus
```

```
| orsy  
  
<multiplicative-operator> ->  
  times  
  | rdiv  
  | idiv  
  | imod  
  | andsy
```

6 Kesimpulan dan Saran

6.1 Kesimpulan

Pada milestone 2 ini berhasil diimplementasikan parser bahasa Arion menggunakan metode Recursive Descent.

Parser mampu melakukan syntax analysis terhadap token hasil lexical analysis dan membentuk Parse Tree yang merepresentasikan struktur sintaks program secara hierarkis. Implementasi parser dilakukan secara modular dengan setiap non-terminal direpresentasikan sebagai fungsi parser tersendiri.

Beberapa fitur yang berhasil diimplementasikan antara lain:

1. Declaration parsing
2. Assignment statement
3. Conditional statement
4. Loop statement
5. Procedure/function declaration dan call
6. Array type dan multidimensional indexing
7. Record type dan field access
8. Expression parsing dengan prioritas operator
9. Parse Tree generation
10. Syntax error handling

Parser juga mampu mendeteksi syntax error dan menghasilkan pesan error yang informatif tanpa menyebabkan program crash. Selain itu, parser tetap dapat membentuk Parse Tree parsial menggunakan error node sehingga proses debugging menjadi lebih mudah.

Berdasarkan hasil pengujian, parser berhasil memproses berbagai jenis program valid maupun invalid.

6.2 Saran

Beberapa pengembangan yang dapat dilakukan pada milestone selanjutnya antara lain:

1. Menambahkan semantic analysis seperti type checking dan scope checking.
2. Mengembangkan Parse Tree menjadi Abstract Syntax Tree (AST) agar struktur program lebih efisien untuk tahap kompilasi berikutnya.
3. Menambahkan mekanisme error recovery yang lebih baik agar parser dapat melanjutkan parsing setelah menemukan syntax error.
4. Menambahkan validasi semantic terhadap procedure/function call dan parameter.
5. Mengoptimalkan struktur parser agar lebih mudah dikembangkan untuk fitur bahasa Arion berikutnya.
6. Mengembangkan intermediate code generation dan interpreter pada milestone selanjutnya.

A Lampiran

A.1 Pranala Ke Repositori

<https://github.com/mhps-null/MDC-Tubes-IF2224-2026>

A.2 Workspace Diagram Transisi DFA

<https://drive.google.com/file/d/1MjCcgpbi4xMZfAtdd9oUyLAfq9YoDl/view?usp=sharing>

A.3 Pembagian Tugas

NIM	Nama	Tugas
13524041	Nathan Adhika Santosa	Laporan, Source Code
13524053	Muhammad Haris Putra Sulastianto	Laporan, Source Code
13524085	Ariel Cornelius Sitorus	Source Code
13524091	Vara Azzara Ramli Pulukadang	Source Code

Table 3: Pembagian Tugas

Referensi

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson, 2 edition, 2006.
- [2] Edunex. Materi 3: Compiler parsing dan analisis semantik, 2026. Diakses: 7 Mei 2026. URL: https://edunexcontentproshot.blob.core.windows.net/edunex/2026/86182-Formal-Language-Theory-and-Automata/407468-Module-13/file/1777512205430_Materi-3-Compiler-Parsing-dan-Analisis-Semantik?sv=2024-11-04&spr=https&st=2026-04-30T01%3A23%3A31Z&se=2028-04-29T01%3A23%3A31Z&sr=b&sp=r&sig=5WQHKv%2BQddhPKZ%2FHRIY2wJaMMEQro%2B13B9zR6pMC%2Byo%3D&rsct=application%2Fpdf.
- [3] GeeksforGeeks. Introduction of parsing ambiguity and parsers — set 1, n.d. Diakses: 4 Mei 2026. URL: <https://www.geeksforgeeks.org/compiler-design/introduction-of-parsing-ambiguity-and-parsers-set-1/>.
- [4] GeeksforGeeks. Introduction to syntax analysis in compiler design, n.d. Diakses: 5 Mei 2026. URL: <https://www.geeksforgeeks.org/compiler-design/introduction-to-syntax-analysis-in-compiler-design/>.
- [5] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson, 3 edition, 2006.